



INDIRA GANDHI NATIONAL OPEN UNIVERSITY

PROJECT PROPOSAL ON

"SKILLSWAP: A LOCAL SKILL EXCHANGE PLATFORM"

Sirjan Lamichhane

Enrollment Number: **239150610**

BCA (6th Semester)

Under the Guidance of

Mr. Trailokya Ojha

Submitted to the School of Computer and Information Sciences in
partial fulfillment of the requirements for the degree of **Bachelors of
Computer Applications**
(BCSP-064)



INDIRA GANDHI NATIONAL OPEN UNIVERSITY
Maidan Garhi
New Delhi — 110068

Table of Content

1.Introduction and Objectives of the Project	1
Introduction	1
Objectives	1
2.Project Category	2
3.Analysis	3
3.1.Data Flow Diagrams (DFDs)	3
Level 0 – Context Diagram	3
Level 1 – Process Decomposition	3
3.2.Entity-Relationship Diagram (ERD)	5
3.3.Database Design	7
4.Complete Structure	8
4.1.Modules & Description	8
4.2.Process Logic	9
4.4.Testing Process	10
4.5.Reports Generation	10
5.Tools / Platform, Hardware & Software Requirements	11
6.Future Scope & Enhancements	12
7.Abstract	13
8.Conclusions	13
9.References	14

1. Introduction and Objectives of the Project

Introduction

In today's rapidly evolving world, individuals possess a wide range of skills that often remain underutilized due to the lack of proper platforms for exchange. Traditional learning methods usually involve monetary transactions, which may not be feasible for everyone. The proposed system, "SkillSwap," aims to create a local skill exchange platform where users can share and acquire skills without financial constraints.

SkillSwap is a web-based platform developed using Python (Flask) and PostgreSQL that enables users to register, list skills they can offer, and specify skills they wish to learn. Based on this information, the system facilitates connections between users for mutual benefit. The platform promotes collaborative learning, community building, and efficient utilization of human resources. It is particularly relevant in developing communities and student environments where access to paid skill-learning resources is limited.

The system addresses the gap between skill availability and skill accessibility by creating a digital marketplace of human talent and knowledge, entirely free of monetary transactions. It builds a self-sustaining learning community where every member is both a teacher and a learner.

Objectives

- To design and develop a web-based skill exchange platform using Python (Flask) and PostgreSQL.
- To enable users to register and manage their personal profiles with offered and desired skills.
- To allow users to list skills they can offer and skills they wish to learn.
- To implement an intelligent matching system that connects users with complementary skills.
- To provide a request and response mechanism for initiating and managing skill exchanges.
- To incorporate a notification system for real-time updates on request status.
- To implement a rating and feedback mechanism to ensure quality and trust within the platform.
- To ensure efficient and secure data management using PostgreSQL.

2. Project Category

SkillSwap is a Web Application that falls under two primary software development categories:

-
- **RDBMS (Relational Database Management System):** The project uses a structured relational database (PostgreSQL) to store and manage user profiles, skills, requests, exchanges, ratings, and notifications with full data integrity through primary and foreign key relationships.
 - **OOPS (Object-Oriented Programming):** The system is designed using OOP principles through Python's class-based architecture, ensuring modular, reusable, and maintainable code. Flask's blueprint-based routing further enforces separation of concerns.

The project is implemented using an MVC (Model-View-Controller) pattern:

- **Model:** PostgreSQL database tables managed via SQLAlchemy ORM or raw psycopg2 queries.
- **View:** HTML/CSS/JavaScript frontend rendered through Jinja2 templates.
- **Controller:** Flask routes and business logic in Python.

3. Analysis

3.1.Data Flow Diagrams (DFDs)

Data Flow Diagrams (DFDs) are graphical representations that depict how data flows through a system. They show the inputs, outputs, storage, and processes of a system at varying levels of detail.

Level 0 – Context Diagram

The Level 0 DFD, also called the Context Diagram, represents the highest-level view of the SkillSwap system. It shows the entire system as a single process and illustrates how it interacts with the primary external entity, the User. The user inputs data into and receives information from the system, and the system communicates with two data stores: D1 (Users & Skills) and D2 (Requests).

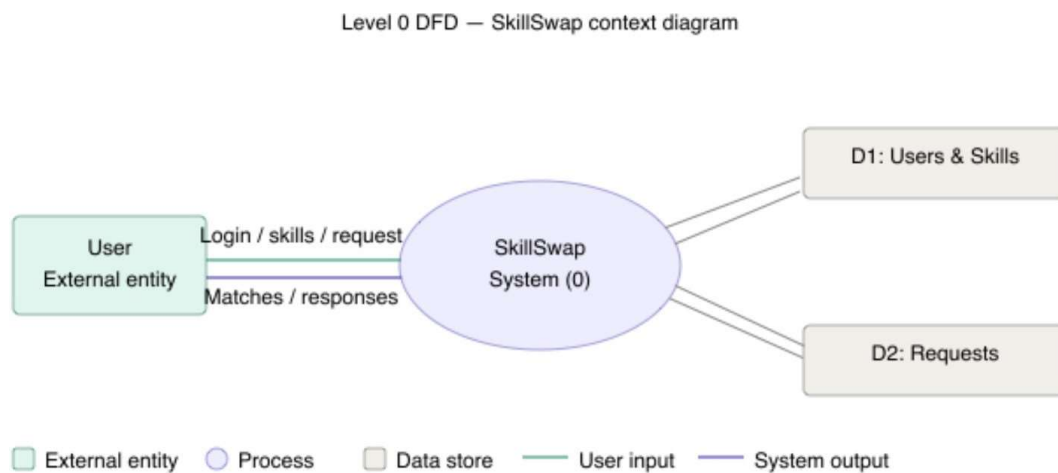


Figure 1: DFD Level 0 – Context Diagram for SkillSwap

Level 1 – Process Decomposition

The Level 1 DFD breaks the SkillSwap system down into four core processes, showing in detail how data flows between the User, each process, and the underlying data stores. The four processes are:

- 1.0 Authentication: Handles user registration and login. Receives credentials from the User and reads/writes user data to D1: Users.
- 2.0 Skill Management: Allows users to add and manage skills. Reads and writes skill and user-skill data across D2: Skills and D3: UserSkills.
- 3.0 Matching: Processes search queries from users and returns matched user profiles by reading from D3: UserSkills.

- 4.0 Request Management: Manages skill exchange requests, reviews, and notifications. Reads/writes to D4: Requests, D5: Reviews, and D6: Notifications.

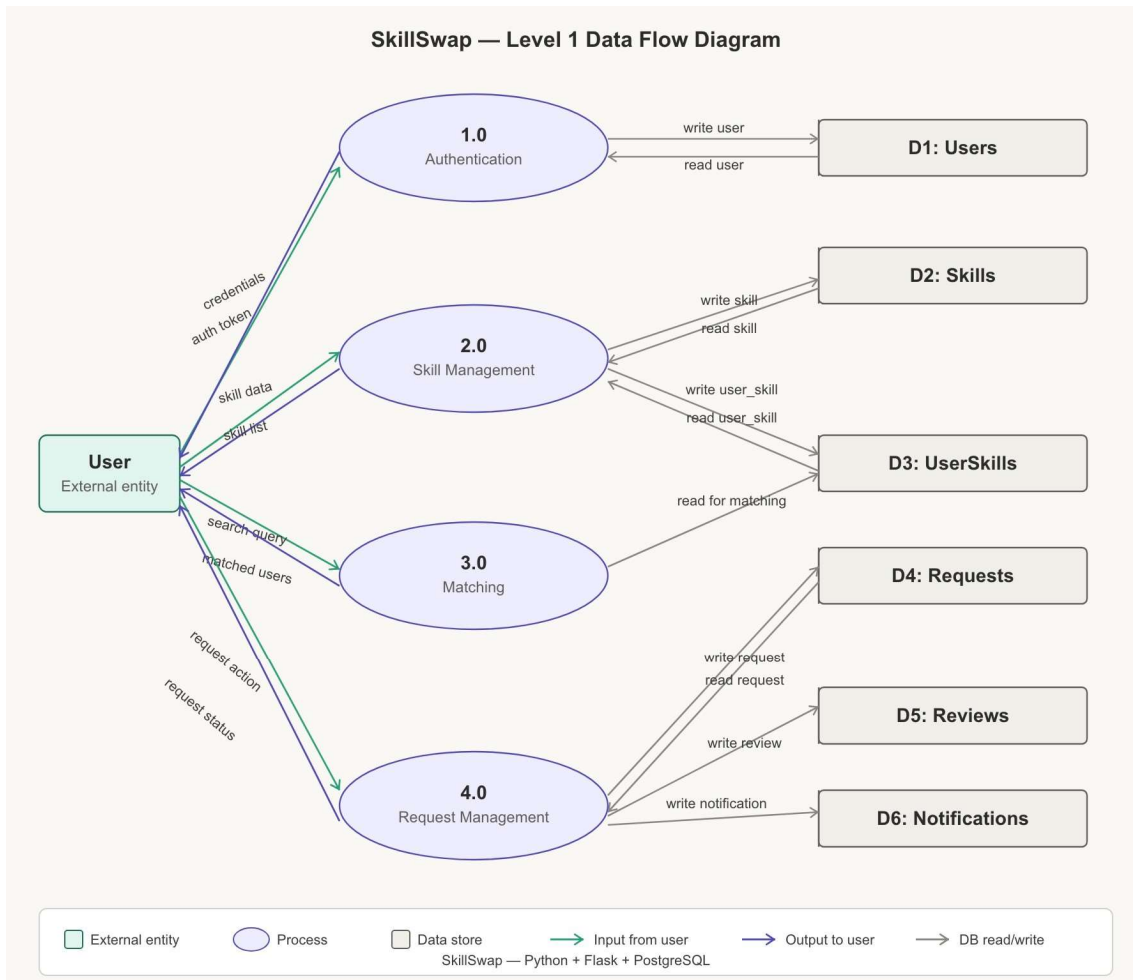


Figure 2: DFD Level 1 – Process Decomposition for SkillSwap

3.2.Entity-Relationship Diagram (ERD)

The Entity-Relationship Diagram (ERD) represents the logical structure of the SkillSwap database, showing entities, their attributes, and the relationships between them. The following entities have been identified:

- User: Represents a registered user of the platform. Attributes: user_id (PK), name, email, password, bio, location, is_active, created_at.
- Skill: Represents a skill available in the system. Attributes: skill_id (PK), skill_name, description, category.
- User_Skills: Junction table linking Users to Skills with type (offer/want) and level. Attributes: id (PK), user_id (FK), skill_id (FK), type, level.
- Request: Represents a skill exchange request between two users. Attributes: request_id (PK), message, status, created_at.
- Exchange: Represents a confirmed skill exchange session. Attributes: exchange_id (PK), request_id (FK), start_date, end_date, status.
- Rating: Stores post-exchange ratings and feedback. Attributes: rating_id (PK), exchange_id (FK), rater_id (FK), rated_user_id (FK), rating, feedback.
- Notifications: Stores platform notifications for users. Attributes: notification_id (PK), user_id (FK), exchange_id (FK), type, is_read, message.
- Message: Represents direct messages between users. Attributes: message_id (PK), sender_id (FK), receiver_id (FK), content, is_read, timestamp.

Key Relationships:

- User HAS many User_Skills (1:N)
- Skill IS ASSOCIATED WITH many User_Skills (1:N)
- User SENDS many Requests (1:N)
- User RECEIVES many Requests (1:N)
- Request BECOMES one Exchange (1:1)
- Request TRIGGERS Notifications (1:N)
- Exchange RECEIVES many Ratings (1:N)
- Exchange SENDS many Messages (1:N)

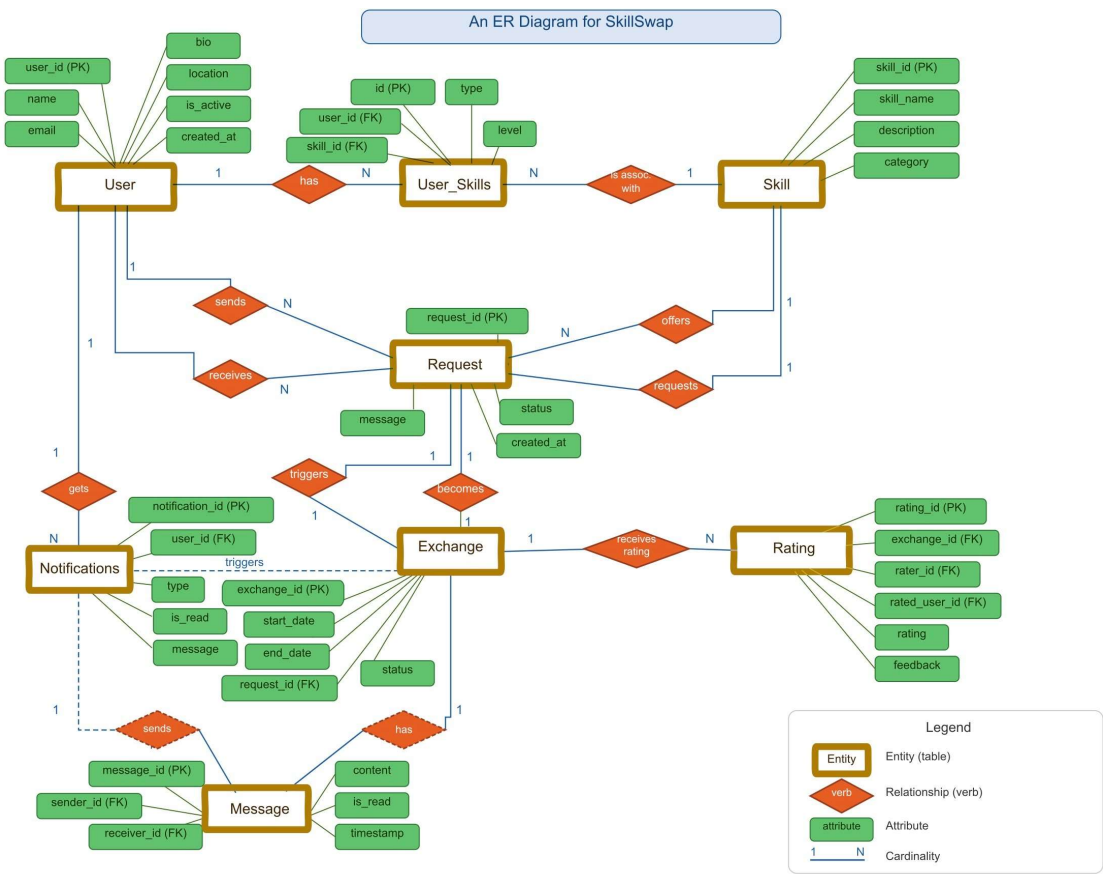


Figure 3: Entity-Relationship Diagram for SkillSwap

3.3.Database Design

The database is designed in PostgreSQL with the following core tables and their structures:

Table Name	Primary	Foreign Keys	Key Attributes
users	user_id	—	name, email, password, bio, location, is_active,
skills	skill_id	—	skill_name, description, category
user_skills	id	user_id, skill_id	type (offer/want), level
requests	request_id	sender_id, receiver_id, offered_skill_id, requested_skill_id	message, status, created_at
exchanges	exchange_id	request_id	start_date, end_date, status
ratings	rating_id	exchange_id, rater_id, rated_user_id	rating, feedback
notifications	notification_id	user_id, exchange_id	type, is_read, message
messages	message_id	sender_id, receiver_id	content, is_read, timestamp

4. Complete Structure

4.1. Modules & Description

Module	Description	Effort Estimation
Project Planning & Analysis	Project Definition, Objectives, Requirement Analysis (DFD, ERD)	1 Week
User Authentication	Registration, Login, Session Management, Role-Based Access Control	1 Week
Profile Management	View/Edit user profiles, manage offered and desired skills	1 Week
Skill Management	Add, update, remove skills; categorize skills by type and level	1 Week
Matching Module	Search and match users based on complementary skill sets	2 Weeks
Request Management	Send, accept/reject skill exchange requests; manage request lifecycle	2 Weeks
Exchange Management	Track active exchanges, set dates, update exchange status	1 Week
Notification System	Real-time platform notifications for requests and exchange updates	1 Week
Rating & Feedback	Post-exchange ratings and written feedback for users	1 Week
Messaging System	Basic in-platform direct messaging between matched users	1 Week
Testing & QA	Unit, Integration, and User Acceptance Testing	1 Week
Deployment & Documentation	Deployment to server, technical documentation, user guide	2 Weeks

4.2. Data Structures

Core data structures used in the SkillSwap system:

- User: { user_id, name, email, password_hash, bio, location, is_active, created_at }

- Skill: { skill_id, skill_name, description, category }
- UserSkill: { id, user_id (FK), skill_id (FK), type: ['offer'|'want'], level: ['beginner'|'intermediate'|'advanced'] }
- Request: { request_id, sender_id (FK), receiver_id (FK), offered_skill_id (FK), requested_skill_id (FK), message, status: ['pending'|'accepted'|'rejected'], created_at }
- Exchange: { exchange_id, request_id (FK), start_date, end_date, status: ['active'|'completed'|'cancelled'] }
- Rating: { rating_id, exchange_id (FK), rater_id (FK), rated_user_id (FK), rating: [1-5], feedback }
- Notification: { notification_id, user_id (FK), exchange_id (FK), type, is_read, message }

4.3.Process Logic

The key business processes in SkillSwap are as follows:

1. User Registration & Login:

User fills registration form → System validates input → Password is hashed and stored → User is redirected to profile setup → JWT/session token is issued for authenticated access.

2. Skill Listing:

User selects skills to offer and skills to learn → System stores entries in user_skills table with type field → Skills appear on user profile.

3. Matching:

User submits a search query → System queries user_skills to find users whose offered skills match the searcher's wanted skills (and vice versa) → Matched profiles are returned as a list.

4. Request & Exchange Flow:

Sender selects a matched user and sends a skill exchange request → System creates a record in requests table → Receiver gets a notification → Receiver accepts or rejects → On acceptance, a new exchange record is created → Both parties complete the exchange → Both rate each other.

4.4. Testing Process

- Unit Testing: Individual functions and modules are tested in isolation using Python's unittest framework. Tests cover authentication logic, skill matching algorithm, request lifecycle, and data validation.
- Integration Testing: End-to-end testing of connected modules, verifying that Flask routes correctly process user input, interact with the PostgreSQL database, and return appropriate responses to the frontend.
- User Acceptance Testing (UAT): The system is tested by a small group of sample users who verify that the platform meets their needs, flows are intuitive, and all features work as expected.

4.5. Reports Generation

- User Activity Report: Lists active users, their offered/wanted skills, and exchange history.
- Exchange Summary Report: Overview of completed, pending, and cancelled exchanges.
- Rating & Feedback Report: Aggregated ratings per user for platform trust assessment.
- Skill Demand Report: Most requested skills on the platform for community insights.

5. Tools / Platform, Hardware & Software Requirements

5.1. Software Requirements

Category	Technology / Tool
Programming Language	Python 3.x
Web Framework	Flask (Python micro-framework)
Database	PostgreSQL 15+
ORM / DB Connector	psycopg2 / SQLAlchemy
Frontend	HTML5, CSS3, JavaScript (Jinja2 templates)
Authentication	Flask-Login, Werkzeug (password hashing)
IDE / Code Editor	Visual Studio Code
DB Administration	pgAdmin 4
Version Control	Git / GitHub
Deployment	Localhost

5.2. Minimum Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 or equivalent (1 GHz+)
RAM	2 GB (4 GB recommended)
Storage	10 GB free HDD/SSD space
Operating System	Windows 10 / macOS / Linux (Ubuntu 20.04+)
Internet	Required for initial package installation and deployment

6. Future Scope & Enhancements

While the current scope of SkillSwap covers essential features for a functional skill exchange platform, the following enhancements are envisioned for future development:

-
- **Mobile Application:** Develop native Android and iOS applications for wider accessibility and better user experience on mobile devices.
 - **Real-Time Chat System:** Integrate WebSocket-based real-time chat using Flask-SocketIO or an external service like Firebase to enable live communication between matched users.
 - **AI-Based Skill Recommendations:** Implement a machine learning recommendation engine that suggests relevant users and skills based on user behavior and matching patterns.
 - **Video/Audio Sessions:** Integration with WebRTC or Zoom API to support virtual skill exchange sessions directly within the platform.
 - **Gamification:** Introduce badges, points, and leaderboards to encourage engagement and reward active community members.
 - **Skill Verification System:** Allow users to showcase certifications or complete skill assessments to validate their expertise, building trust on the platform.
 - **Community Forums:** Add discussion boards and Q&A sections for collaborative learning beyond one-on-one exchanges.
 - **Multi-Language Support:** Localization for regional languages (e.g., Nepali, Hindi) to serve a broader local audience.

7. Abstract

SkillSwap is a web-based skill exchange platform developed using Python (Flask) and PostgreSQL, designed to connect individuals who wish to share and acquire skills without monetary transactions. In a world where quality skill-learning resources are often tied to financial accessibility, SkillSwap provides an open, community-driven alternative that leverages the diverse talents within a local population.

The system enables users to register, build detailed profiles, and list skills they can offer alongside skills they wish to learn. A core matching engine identifies users with complementary skill sets, enabling them to connect, send exchange requests, and collaborate through structured skill exchange sessions. The platform also features a rating and feedback system to foster trust and accountability within the community.

Built on a structured software engineering foundation, incorporating Data Flow Diagrams (DFDs) at Level 0 and Level 1, as well as a comprehensive Entity-Relationship Diagram (ERD), SkillSwap follows the Agile development methodology for iterative, test-driven delivery. The system architecture adheres to MVC design principles, ensuring modularity, maintainability, and scalability.

The project demonstrates practical competency in full-stack web development, relational database management, and software engineering processes. Beyond its academic purpose, SkillSwap addresses real-world community needs by creating a sustainable, peer-to-peer learning ecosystem, one that is cost-effective, inclusive, and scalable. Future enhancements such as real-time chat, mobile applications, and AI-based recommendations position SkillSwap for broader impact.

8. Conclusions

The SkillSwap platform represents a meaningful intersection of technology and community development. By providing a structured, web-based environment for skill exchange, it directly addresses the challenge of unequal access to skill-learning resources, enabling individuals to trade knowledge freely and collaboratively.

Through the use of Python (Flask) and PostgreSQL, the system is technically feasible, economically sustainable (relying entirely on open-source tools), and operationally straightforward for non-technical users. The use of structured software engineering methodologies, including DFDs, ERD modeling, and Agile development, ensures that the system is both well-designed and well-documented.

From a practical standpoint, SkillSwap is more than an academic exercise. It is a viable product with real demand in student communities, local neighborhoods, and developing regions where formal learning resources are scarce. With the proposed future enhancements, including mobile support, real-time communication, and AI-driven recommendations, SkillSwap is well-positioned to evolve into a full-scale community learning platform.

In conclusion, SkillSwap is a technically sound, socially relevant, and academically rigorous project that fulfills the requirements of BCSP-064 while delivering real value to its intended user community.

9. References

- Python Software Foundation. Python 3 Official Documentation. <https://docs.python.org/3/>
- Pallets Projects. Flask Documentation (3.x). <https://flask.palletsprojects.com/>
- The PostgreSQL Global Development Group. PostgreSQL 15 Documentation. <https://www.postgresql.org/docs/15/>
- Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill Education.
- Date, C. J. (2003). An Introduction to Database Systems (8th ed.). Addison-Wesley.
- SQLAlchemy Documentation. <https://docs.sqlalchemy.org/>
- Research papers and online resources related to skill-sharing and peer- learning platforms.